

Learning Report – Module 12: Containerization Using Docker

Overview

This module introduced me to Docker, one of the most popular containerization platforms used in modern software development. I learned how Docker helps package applications along with their dependencies into containers, making them portable, lightweight, and easy to deploy across different environments. The module also covered Docker commands, Docker images, Docker Compose, Docker Engine, storage, networking, and an introduction to container orchestration.

Learning Outcomes

After completing this module, I was able to:

- Understand the concept and features of Docker.
- Learn basic Docker commands for managing containers and images.
- Understand Docker images and how they are created.
- Learn the purpose of Docker Compose.
- Understand the architecture of Docker Engine.
- Learn Docker storage and volume management.
- Understand Docker networking concepts.
- Gain basic knowledge of container orchestration and Kubernetes.

Introduction to Docker

Docker is an open-source platform used to develop, package, and run applications inside lightweight containers. Containers include the application along with all its required libraries and dependencies, ensuring that the application works consistently across different environments.

Features of Docker

- Lightweight and portable containers.
- Faster application deployment.
- Platform independence.
- Efficient resource utilization.

- Easy scalability.
- Simplified application management.

https://www.geeksforgeeks.org/devops/docker-tutorial/

Docker Tutorial
Last Updated : 19 Feb, 2026

Docker is a tool that simplifies the process of developing, packaging, and deploying applications. It solves the “works on my machine” problem by ensuring code runs identically across environments.

- Docker allows you to build, package, and run applications inside containers
- Containers are lightweight piece of software that contains all the code, libraries, and dependencies that the application needs to run.
- Containers share the host operating system’s kernel but keep applications and their dependencies isolated. This makes containers faster to start, less resource-hungry, and portable across environments.

Installing Docker

Download → Install → Run

Basic Docker Commands

I learned several commonly used Docker commands:

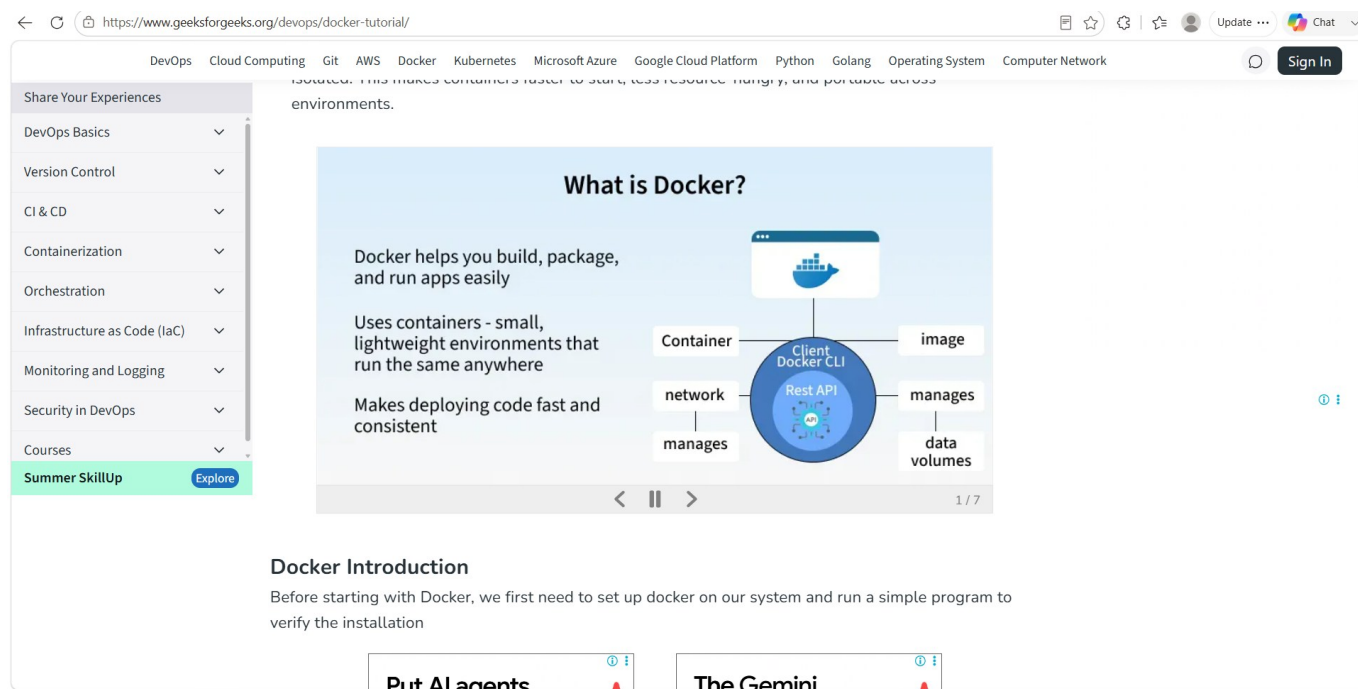
- `docker run` – Creates and starts a container.
- `docker ps` – Lists running containers.
- `docker stop` – Stops a running container.
- `docker rm` – Removes a container.
- `docker images` – Displays available Docker images.
- `docker rmi` – Removes a Docker image.
- `docker pull` – Downloads an image from Docker Hub.
- `docker exec` – Executes commands inside a running container.

These commands are essential for managing Docker containers during development.

Docker Run

The docker run command is used to create and start containers. During this module, I learned how to:

- Run a container using an image.
- Assign a custom name to a container.
- Run containers in detached (background) mode.
- Run containers interactively.
- Publish container ports to access applications.
- Automatically remove containers after execution.



Docker Images

A Docker image is a read-only template used to create containers. Images contain the application, libraries, runtime, and configuration files required to run the application.

I also learned about:

- Base images and parent images.
- Image layers.
- Container layer.

- Dockerfile for building custom images.
- Docker Build Context.
- Docker Hub and other container registries.

Docker Compose

Docker Compose is a tool used to define and manage multi-container Docker applications using a YAML configuration file.

Benefits of Docker Compose

- Simplifies multi-container application management.
- Starts multiple services with a single command.
- Easy configuration using YAML files.
- Improves development and testing workflows.

I also learned the basic Docker Compose commands and how Compose files are structured.

The screenshot shows the GeeksforGeeks website with the URL <https://www.geeksforgeeks.org/devops/docker-tutorial/>. The page is titled "Docker Networking" and "Docker Registry". The sidebar on the left lists various topics under "Share Your Experiences", including DevOps Basics, Version Control, CI & CD, Containerization, Orchestration, Infrastructure as Code (IaC), Monitoring and Logging, Security in DevOps, and Courses. The main content area is divided into sections: "Docker Networking" (describing mechanisms and technologies for communication between containers), "Docker Registry" (describing a central repository for storing and managing Docker images), and "Docker Volumes" (describing a way to store data outside of a container's filesystem). The right sidebar shows "Upcoming Courses" with details for DevOps Course, Java Backend, and MERN Full Stack, along with "Fresher Jobs" for Mobile App Security Intern, Backend Developer Intern, and Odoo Functional Consultant.

Docker Engine

Docker Engine is the core component responsible for building and running Docker containers.

It consists of:

- **Docker CLI** – Command-line interface used to execute Docker commands.
- **Docker Daemon** – Background service that manages Docker objects.
- **REST API** – Allows communication between Docker clients and the Docker daemon.

Together, these components enable users to create and manage containers efficiently.

Docker Storage

Docker provides storage options to preserve data even when containers are removed.

During this module, I learned about:

- Storage drivers.
- Data volumes.
- Creating Docker volumes.
- Listing available volumes.
- Managing persistent storage for containers.

Volumes are useful for storing application data independently of container lifecycles.

The screenshot displays the GeeksforGeeks Docker tutorial page. The left sidebar contains a navigation menu with categories such as DevOps Basics, Version Control, CI & CD, Containerization, Orchestration, Infrastructure as Code (IaC), Monitoring and Logging, Security in DevOps, and Courses. The main content area is titled 'Docker Swarm' and includes a description of Docker Swarm as a tool for clustering and orchestrating multiple containers. It also lists links to 'Docker Swarm', 'Docker - Security Best Practices', and 'How To Use Docker Security Tools To Secure Docker Container Images'. Below this, there is a section titled 'Docker Vs Kubernetes' with bullet points: 'Docker -> creates and runs containers.' and 'Kubernetes -> manages and orchestrates containers at scale.' The right sidebar features 'Upcoming Courses' with titles like 'DevOps Course with...', 'Java Backend...', and 'MERN Full Stack...', and 'Fresher Jobs' with titles like 'Mobile App Security Intern', 'Backend Developer Intern', and 'Odoo Functional Consultan...'.

Docker Networking

Docker networking enables communication between containers and external systems.

I learned about:

- Default Docker networks.
- Listing Docker networks.
- Inspecting existing networks.
- Creating custom Docker networks.

Networking allows multiple containers to communicate securely within an application.

Container Orchestration

Container orchestration is the process of managing multiple containers automatically.

I learned that orchestration helps with:

- Automatic deployment.
- Scaling applications.
- Load balancing.
- Monitoring container health.
- Managing container failures.

The module also introduced Kubernetes, the most widely used container orchestration platform, and explained how it differs from Docker. While Docker is mainly used to create and run containers, Kubernetes is used to manage large numbers of containers in production environments.

Conclusion

This module provided a solid understanding of Docker and containerization concepts. I learned how Docker simplifies application deployment by packaging applications into containers and explored essential Docker commands, Docker images, Docker Compose, Docker Engine, storage, and networking. I also gained an introduction to container orchestration and Kubernetes for managing containerized applications at scale. Overall, this module enhanced my understanding of modern application deployment practices and the role of Docker in DevOps and cloud-based software development.